

alerts

Overview

`alerts` is a **device-emitted** event that delivers structured, categorized alert information — the reader publishes it **autonomously** when a monitored condition changes state or crosses a threshold; **no command triggers it**. Each event is a fixed **envelope** (`type` , `timestamp` , `state` , `id` , `priority`) plus a category-specific `alertDetails` body selected by `id` . It is read-only telemetry and carries no credentials.

The same envelope carries two message classes, distinguished by `type` :

- `ALERT` — an autonomous monitoring alert (battery, power, network).
- `NOTIFICATION` — a command side-effect: `FILE_DOWNLOAD` (see `install_certificate`) and the `FIRMWARE_UPDATE` start (see `set_os`).

This page documents the full envelope and **all seven** `alertDetails` **branch keys**. `config_events` toggles (`battery` , `power` , `network` , `firmwareUpdate` , `fileDownload`) govern which alerts publish.

The `id` enum has five categories, all emitted on current V1.1 firmware: `BATTERY` , `POWER` , `NETWORK_EVENT` , `FIRMWARE_UPDATE` , and `FILE_DOWNLOAD` . The schema also declares a `temperatureInfo` branch, but there is no `TEMPERATURE` id and no temperature alert is emitted on current firmware. See [Events](#).

Details

Property	Value
Event Type	Alert / notification (structured, categorized)
Trigger	State change / threshold (autonomous, on a monitored condition change or threshold crossing)
Enabled By	<code>config_events</code> with per-category <code>eventConfiguration</code> flags (<code>battery</code> , <code>power</code> , <code>network</code> , <code>firmwareUpdate</code> , <code>fileDownload</code>), applied on <code>reboot</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)

Property	Value
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>heartBeatEVT</code> , <code>dataEVT</code> , <code>mqttConnEVT</code>

When to Use

Subscribe to `alerts` when you want the reader to **tell you** about a monitored condition rather than asking for it — there is **no command that returns this data on demand** (no polling twin). Reach for it when you need to:

- **React to device-health changes as they happen** — battery state and health (`BATTERY`), power-source and power-mode transitions (`POWER`), and network interface connect/disconnect (`NETWORK_EVENT`).
- **Track long-running command side-effects** — the `FIRMWARE_UPDATE` stream during a `set_os` flash (a `type: NOTIFICATION` start, then in-progress notifications) and the `FILE_DOWNLOAD` result during an `install_certificate` transfer (`type: NOTIFICATION`), all on this same envelope.
- **Triage by severity** — the `priority` field (`CRITICAL` ... `LOW`) lets you route alerts without inspecting each branch body.

Enable only the categories you consume with `config_events` — a disabled category never publishes, so leaving one off is how you silence it. For a periodic health snapshot rather than change-driven alerts, use `heartBeatEVT`.

Topic Mapping

`alerts` is device-emitted and flows on the **management event topic** — it has no `cmd` / `resp` topics because no command produces it.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	The same event topic that carries <code>heartBeatEVT</code> and <code>NOTIFICATION</code> -type events.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the

reader's serial number, which the reader appends itself in its model-prefixed form (for example RFD40-212735201D0053). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `type` (event) — message class: `ALERT` (autonomous monitoring alert), `NOTIFICATION` (command side-effect), or the reserved `EVENT`. Route on this to separate alerts from notifications.
- `timestamp` (event) — ISO-8601 (`format: date-time`) instant the alert was generated.
- `state` (event) — `SET` (condition active/asserted), `CLEAR` (resolved), or `ONESHOT` (one-time, no matching `CLEAR`). Pair with `id` to track a condition's lifecycle.
- `id` (event) — the alert category (`BATTERY` , `POWER` , `NETWORK_EVENT` , `FIRMWARE_UPDATE` , `FILE_DOWNLOAD`); selects which `alertDetails` branch is populated.
- `priority` (event) — severity, `CRITICAL` / `HIGH` / `LOW` , for triage without inspecting the branch body.
- `alertDetails` (event) — the category-specific body (seven declared branch keys); exactly one branch is populated, matching `id` . Optional — not in the envelope `required` set.
- `config_events` **category toggles** (request) — the emission gate: the per-category `eventConfiguration` flags (`battery` , `power` , `network` , `firmwareUpdate` , `fileDownload`) set with `config_events` govern whether each `id` publishes at all. A disabled category is silent.

Event Payload

The envelope (`type` , `timestamp` , `state` , `id` , `priority`) is always present and **required**; `alertDetails` is optional and carries exactly one branch matching `id` . Pick an **alert category** below — the Example and Schema update together to show that category's `alertDetails` branch. Use the **Schema** tab for the field reference and the **Example** tab for the live payload.

Alert category

Battery alert



Event payload

Example: BATTERY (live)

```
1  {
2    "type": "ALERT",
3    "timestamp": "2026-04-29T12:33:34.279Z",
4    "state": "ONESHOT",
5    "id": "BATTERY",
6    "priority": "LOW",
7    "alertDetails": {
8      "batteryAlert": {
9        "status": "CHARGING",
10       "stateOfHealth": "GOOD",
11       "chargePercentage": 100
12     }
13   }
14 }
```

Event payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	type *	enum	EVENT NOTIFICATION ALERT	ALERT
2	timestamp *	string	date-time	2026-06-17T10:16:34.793Z
3	state *	enum	SET CLEAR ONESHOT	ONESHOT
4	id *	enum	BATTERY FILE_DOWNLOAD FIRMWARE_UPDATE NETWORK_EVENT POWER	BATTERY
5	priority *	enum	CRITICAL HIGH LOW	CRITICAL

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
6	alertDetails	object	—	—
7	fwUpdateStatus	object	—	—
8	updateStatus *	enum	Starting FW update FW update success updating failed skipping	FW update success
9	overallProgress	number	—	20
10	stage	string	—	updating scanner fw
11	downloadInfo	object	—	—
12	fileType *	enum	FILE_TYPE_FW FILE_TYPE_CERT	FILE_TYPE_FW
13	fileName *	string	—	mqtt_ca_cert
14	status *	enum	DOWNLOAD SUCCESS DOWNLOAD FAIL SAVE FAIL SAVE SUCCESS	DOWNLOAD SUCCESS

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
15	temperatureInfo	object	—	—
16	temperature *	number	—	—
17	pressure *	number	—	—
18	ambient *	number	—	—
19	networkInfo	object	—	—
20	networkInterface *	object	—	—
21	ethStatus	array	—	—
22	interface *	string	—	eth0
23	status *	enum	CONNECTED DISCONNECTED	CONNECTED

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
24	linkStatus	enum	UP DOWN	UP
25	linkSpeed	string	—	—
26	ipv4Address	string	—	192.168.0.165
27	ipv6Address	string	—	fe80::f299:9d51:864c:cff
28	wifiStatus	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
29	interface	string	—	wlan0
30	status	enum	CONNECTED DISCONNECTED	CONNECTED
31	ssid	string	—	IOT_TESTAP
32	ipV4Address	string	—	192.168.0.145
33	ipV6Address	string	—	fe80::f299:9d51:86c:cff14

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
34	btRemot eDevice	object	—	—
35	dev ice Nam e *	string	—	—
36	dev ice * Mac	string	—	00:11:22:A3:B1:D2
37	ntp	object	—	—
38	offset	number	—	—
39	reach	number	—	—
40	powerEvent	object	—	—
41	powerS ource *	enum	BATTERY USB CRADLE WALL_CHARGER	BATTERY
42	lldp	enum	success failed negotiating	—
43	powerM ode *	enum	ACTIVE LOWPOWER	LOWPOWER
44	batteryAle rt	object	—	—
45	status *	enum	LOW CRITICAL CHARGING FULL HIGH	LOW

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
46	stateOfHealth *	enum	GOOD CRITICAL MODERATE	GOOD
47	chargePercentage *	integer	max 100	100

`state` tells you whether a condition is active or resolved: `SET` = active/asserted, `CLEAR` = resolved, `ONESHOT` = a one-time event with no persistent state (it fires once and has no matching `CLEAR`). Every example on this page is `ONESHOT`, and so was every firmware-update notification recorded during one `set_os` run.

Event Semantics

Emission

The per-category `eventConfiguration` toggles you set with `config_events` govern which `id` categories publish: `battery` → `BATTERY`, `power` → `POWER`, `network` → `NETWORK_EVENT`, `firmwareUpdate` → `FIRMWARE_UPDATE`, and `fileDownload` → `FILE_DOWNLOAD`. A disabled category never publishes, so the *absence* of an alert is not proof of a healthy state. The `FIRMWARE_UPDATE` *start* and the `FILE_DOWNLOAD` result arrive as `type: NOTIFICATION` command side-effects (from `set_os` and `install_certificate`); the `BATTERY` / `POWER` / `NETWORK_EVENT` categories are autonomous `type: ALERT` events. The in-progress `FIRMWARE_UPDATE` events recorded during one `set_os` run were also `NOTIFICATION`s. The `config_events` schema has a `temperature` toggle, but there is no `TEMPERATURE` `id` and no temperature alert is emitted, so enabling it has no effect.

Branches

`alertDetails` declares seven branch keys and carries one of them; `id` selects the populated key. Two keys have no `id` value that selects them:

<code>id</code>	<code>alertDetails</code> key	Body fields	Emitted on V1.1
<code>BATTERY</code>	<code>batteryAlert</code>	<code>status</code> , <code>stateOfHealth</code> , <code>chargePercentage</code>	✓

id	alertDetails key	Body fields	Emitted on V1.1
POWER	powerEvent	powerSource , powerMode , (lldp)	✓
NETWORK_EVENT	networkInfo	networkInterface.{ethStatus[], wifiStatus[]} , btRemoteDevice	✓
FIRMWARE_UPDATE	fwUpdateStatus	updateStatus , overallProgress , stage	✓
FILE_DOWNLOAD	downloadInfo	fileType , fileName , status	✓
— (no TEMPERATURE id)	temperatureInfo	nge , pa , ambient	⚠ not emitted
— (no matching id)	ntp	offset , reach	⚠ no example

Field Support

- **type distinguishes alerts from notifications.** `type: ALERT` is an autonomous monitoring alert; `type: NOTIFICATION` is a command side-effect — `FILE_DOWNLOAD` (during `install_certificate` / firmware download) and the `FIRMWARE_UPDATE` *start* (during `set_os`). `EVENT` is in the enum but was not observed. All three share this one envelope.
- `FILE_DOWNLOAD` `fileName` **is the stored basename.** `downloadInfo.fileName` is the name the reader stores internally — the URL basename **stem** (e.g. a source `.../mqtt_ca_cert.pem` yields `fileName: "mqtt_ca_cert"`), not your request name. Do not expect a 1:1 match to a `get_installed_certificates` entry name without the source URLs.
- **The interface key is `interface` on these items.** Each `ethStatus` / `wifiStatus` item names its interface with the key `interface` (`"interface": "eth0"`), while the Wi-Fi and Ethernet commands and their read-backs use `interfaceName` for the same concept. The key is required: an `ethStatus` item requires `interface` , `status` , `linkStatus` and `ipV4Address` , a `wifiStatus` item `interface` , `status` , `ssid` and `ipV4Address` .
- **alertDetails is optional; envelope is required.** The envelope `required` set is `[type, timestamp, state, id, priority]`; `alertDetails` is not required. Validate the envelope strictly.
- **batteryAlert.stateOfHealth is the same health grade as `get_status`.** The value set here is `GOOD` / `CRITICAL` / `MODERATE` , and `get_status` reports `deviceStatus.batteryStatus.stateOfHealth` from that same enum. One grade, so nothing needs translating when you share battery logic between the command and the event stream. The battery's charge state is the separate `status` field. See [get_status](#).

Examples

Every contract example is in the **Alert category** dropdown under [Event Payload](#), and all of them are the reader's own live captures. For the `FIRMWARE_UPDATE` category, the dropdown shows the `NOTIFICATION` that starts an update (from `set_os`); the in-progress notifications that follow are under `set_os` [Progress](#). The two `FILE_DOWNLOAD` options come from an `install_certificate` transfer: one reports the download, the other the save.

Notes and Caveats

- **Validate against the alert envelope on this page.** It is the shape the reader publishes on the management event topic.
- **No response envelope.** `alerts` is a device-emitted event, so it carries **no** `response` / `code` — the command response-code table does not apply, and no response code is defined for any event.

`alerts` is **read-only telemetry** with no credentials. Remember emission is **gated by** `config_events` (see [Event Semantics](#)). For the concept-level view, see [Alerts](#) and [Event Management](#).

Related:  `config_events` ·  `set_os` ·  `install_certificate` ·  `heartBeatEVT` ·  [Events](#) ·  [Alerts](#) ·  [Events](#)